



DALARNA
UNIVERSITY

Degree Thesis

Master's level (Second cycle)

SpySGX: Automatic Reversing of Secrets from Memory Traces in SGX-Enclaves

Author: Youheng Lü
School: Dalarna University
Supervisor: Ilias Thomas and Michael Schwarz
Examiner: Moudud Alam
Subject/main field of study: Microdata Analysis
Course code: MI4001
Credits: 30 ECTS
Date of examination: 2023-06-08

At Dalarna University it is possible to publish the student thesis in full text in DiVA. The publishing is Open Access, which means the work will be freely accessible to read and download on the internet. This will significantly increase the dissemination and visibility of the student thesis.

Open Access is becoming the standard route for spreading scientific and academic information on the internet. Dalarna University recommends that both researchers as well as students publish their work Open Access.

I give my/we give our consent for full text publishing (freely accessible on the internet, Open Access):

Yes ☒

No ☐

Abstract: Intel Software Guard Extensions (SGX) enable the creation of enclaves for secure deployment of sensitive data in cloud computing. However, memory-based side-channel attacks pose a threat to SGX enclaves, compromising their confidentiality. Although enclave library developers are responsible for addressing side-channel attacks, writing side-channel free code is challenging.

We identified a research gap in the verification step, assessing whether a potential side-channel vulnerability is strong enough to leak the secret. To address this, we propose *SpySGX*, a framework combining symbolic execution and data analysis to automatically leak secrets from enclave Libraries. Unlike existing approaches, *SpySGX* does not require a specific type of memory-access patterns and can be applied to a broader set of enclave Libraries. Additionally, we explore the effectiveness of limiting memory-accesses to mitigate side-channel attacks. We develop two approaches to handle incomplete memory traces and demonstrate their efficacy through testing on two enclave Libraries.

Our results show that *SpySGX* consistently leaks the complete secret given the full trace, verifying the existence of a side-channel vulnerability. Furthermore, our missing instruction heuristic successfully recovers the complete secret in 98 % of cases with 25 % trace incompleteness. For data-access traces, our dynamic time-warping approach demonstrates near-perfect imputation. These findings emphasize the insufficiency of limiting memory-accesses in preventing memory-based side-channel attacks.

Contents

1	Introduction	5
1.1	SGX Memory-based Side-Channel Attacks	6
1.2	Current State of Defenses	6
1.3	Our Contributions	8
2	Background	11
2.1	Intel Software Guard Extensions (SGX)	11
2.1.1	SGX Terminology and Concepts	13
2.2	Side-channel Attacks on Memory Accesses	14
2.2.1	Instruction Access Attacks	14
2.2.2	Data Access Attacks	15
2.2.3	SGX Memory-based Side-Channel Attacks	15
2.3	Emulation	16
2.4	Symbolic Execution	17
2.5	Dynamic Time Warping	18
3	Data Description	19
3.1	Data Collection	19
3.2	Target Enclave Library Description	20
3.2.1	Square and Multiply	20
3.2.2	S-Box Lookup	21
3.3	Data Overview	22
3.3.1	Square and Multiply Data	22
3.3.2	S-Box Lookup Data	25
3.3.3	Missing Data	25
4	Methodology	27
4.1	Symbolic Instruction-Access	27
4.2	Symbolic Data Access	28
4.3	Dealing With Missing Data	30

4.3.1	Missing data heuristic	30
4.3.2	Imputing missing data with dynamic time warping	31
4.4	Experimental Setup	31
4.4.1	Missing Instructions-Accesses	33
4.4.2	Missing Data-Accesses	33
5	Results	35
5.1	Symbolic Instruction Access	35
5.2	Symbolic Data Access	37
5.3	Dealing With Missing Data	39
6	Discussion	41
6.1	Implications	41
6.2	Limitations and Further Work	42
6.3	Related Work	43
7	Conclusions	45

Chapter 1

Introduction

Various software applications handle sensitive data such as passwords, customer information, and deep-learning model parameters, necessitating strict access control. However, a significant vulnerability arises in cloud computing environments where administrative users, including the cloud provider, possess complete access to the operating system and all running processes. This lack of protection leaves sensitive data exposed if the system is compromised. To address this issue, Intel introduced Intel Software Guard Extensions (SGX) in 2015. SGX offers an additional set of CPU instructions that enable the creation of enclaves-secure memory regions shielded by hardware. Enclaves claim to provide robust confidentiality and integrity, safeguarding data even in the presence of compromised operating systems [1].

Despite the security promises of SGX-enclaves, their confidentiality can be compromised by memory-based side-channel attacks, as discussed in section 1.1. Existing defenses against such attacks are examined in section 1.2. However, our contribution lies in addressing a research gap in the verification step, assessing whether a potential side-channel vulnerability is strong enough to leak the secret. In section 1.3, we present *SpySGX*, a novel framework that combines symbolic execution and data analysis to automatically leak secrets from enclave Libraries. *SpySGX* is applicable to a wide range of enclave library functions, offering a valuable tool for verifying potential side-channel vulnerabilities and assisting in the development of secure enclaves.

The thesis is structured as follows: In chapter 2, we give an overview of background information explaining various concepts used in this thesis. In chapter 3, we describe how we collected the traces for our analysis and describe the data. In chapter 4, we describe our methodology and how we combine data science and symbolic execution to reverse the secret. In chapter 5, we present the results of our analysis. In chapter 6, we discuss the results and limitations, propose future work,

and highlight the difference between our work and related work. In chapter 7, we present a summary and conclusions of our work.

1.1 SGX Memory-based Side-Channel Attacks

Multiple categories of attacks [2] have breached the confidentiality of SGX-enclaves. One category of attacks is side-channel attacks [3], which can make conclusions about unknown information by observing potential side effects. One subcategory of side-channel attacks infers sensitive data via knowledge of memory-accesses [4, 5, 3]. While the precise memory-access is not visible to the attacker, an attacker can observe side-effects like page faults [4] or timing differences [5] that are correlated to the memory-accesses. These side effects can be used to infer which memory-accesses have occurred. If the enclave contains a correlation between the memory-accesses and confidential data, an attacker can make inferences about the confidential data. A real-life analogy for this attack would be: We observe that the stove is still hot (side-effect) and conclude that someone has used the stove recently (memory-access). With this information, we can conclude that our roommate has cooked and therefore skipped their diet (secret).

1.2 Current State of Defenses

Intel officially states [6] that defending against side-channel attacks is the responsibility of the enclave Library developers, not Intel. Developers are supposed to write code that avoids any correlation between memory-accesses and secrets. [7] Avoiding correlations is difficult in practice, and even vetted applications like the official GNU cryptographic library Libgcrypt [8] were vulnerable to a memory-based side-channel attack [9] that leaked the encryption key. GNU packages are in the majority of Linux distributions, and Linux, in turn, makes up 38.5 % of public web servers [10]. Another example is WolfSSL [11], a FIPS-certified cryptographic library where researchers also demonstrated that secret encryption keys could be leaked [3]. Given that even vetted cryptographic libraries are vulnerable to side-channel attacks, it is likely that other libraries are also still vulnerable.

To fix more existing vulnerabilities, we need a pipeline to automatically detect and fix these vulnerabilities, as shown in Figure 1.1. Currently, there are multiple approaches to detecting potential vulnerabilities. Tools like DATA [12], STACCO [13], MicroWalk [14] and ANAPLEPS [15] generate multiple traces with different inputs and check whether the traces are correlated with the secret. Another approach called ENCIDER [16] uses symbolic execution and taint tracking to detect

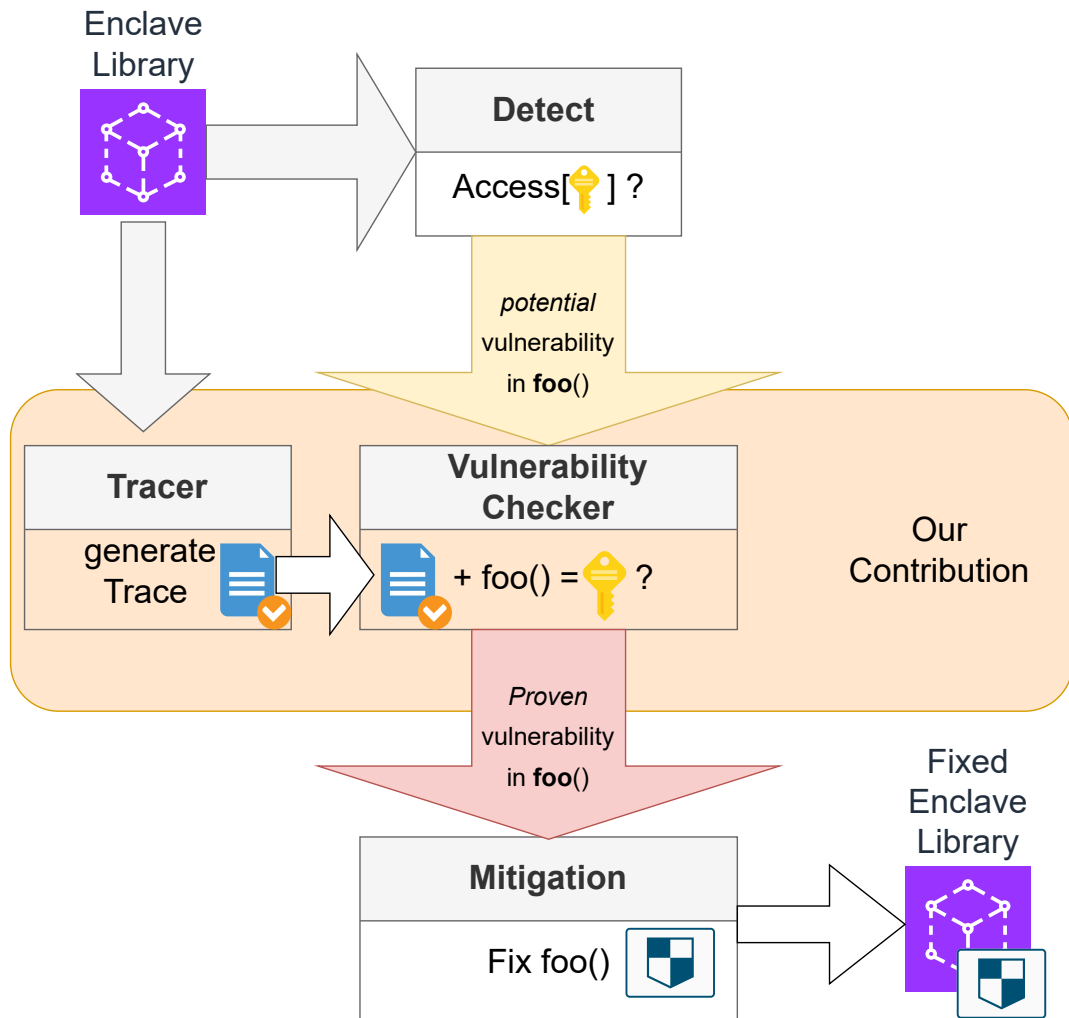


Figure 1.1: Overview of automatically protecting an enclave library from memory-based side-channel attacks. The first step is to *detect* whether the library contains any secret-dependent memory-accesses. If a potential vulnerability is detected, the next step *check* whether it is a real vulnerability. The check is done by collecting a memory-access trace and checking whether the secret can be leaked from the trace. If the library is proven to be vulnerable, the last step is to *fix* the vulnerability. We detected a research gap in the *check* step and developed *SpySGX* that can collect a *trace* and check whether the secret can be leaked from the trace.

potential secret-dependent memory-accesses. While these approaches can detect potential vulnerabilities, they can not distinguish between potential vulnerabilities and false positives. As stated earlier, fixing enclave libraries is difficult, reducing the number of false positives will result in developers spending more time on fixing existing vulnerabilities, which will result in more secure enclave libraries.

One approach to verify whether the potential vulnerability is real is to prove that the secret can be inferred from the memory-access trace. Within memory-access traces, there are two types of information: The first type is the instruction-accesses, which are the addresses of instructions executed by the enclave. The second type is the data-accesses, which are the addresses of data-accessed by the enclave. Depending on the enclave, only one of these types is sufficient to leak the secret. Currently, the only automated approach is Faultmorse [17], which analyses the instruction-access trace for the longest recurring patterns and uses the distance between these patterns to reverse the secret. This approach is limited to enclaves with recurring patterns in their instruction-accesses. It also assumes that the instruction-accesses exhibit a specific repeating pattern, which is not true for all enclaves. Other work has still reverted the secrets manually [4], [5], [3], which is time-consuming and error-prone. Automated reversing secrets remains challenging since every enclave Library and each function has different functionalities, creating different memory-access traces.

Another approach to secure SGX-enclaves is limiting the number of memory-accesses an attacker can observe. To get a complete memory-access trace, attacks like CopyCat [3] use SGX-Step [18] to step through the enclave Library one instruction at a time. The mitigation that is currently in development is Asynchronous Enclave Exit Notify (AEX-Notify) [19]. In theory, this mitigation could disable SGX-Step and prevent the collection of a complete memory-access trace. While this prevents attacks that rely on a complete memory-access trace, it does not prevent attacks that only need parts of the trace [9]. In order to quantify how effective limiting the number of memory-accesses is, we need to quantify how much of the trace is needed to leak the secret.

1.3 Our Contributions

We have identified a research gap in this area in the verification step. The challenge is a typical data science inference problem: Given a dataset (memory-access trace), is the inference of the target variable (secret value) possible? To help answer this question, we have developed *SpySGX*, a framework that automatically checks whether a secret can be leaked from a memory-access trace.

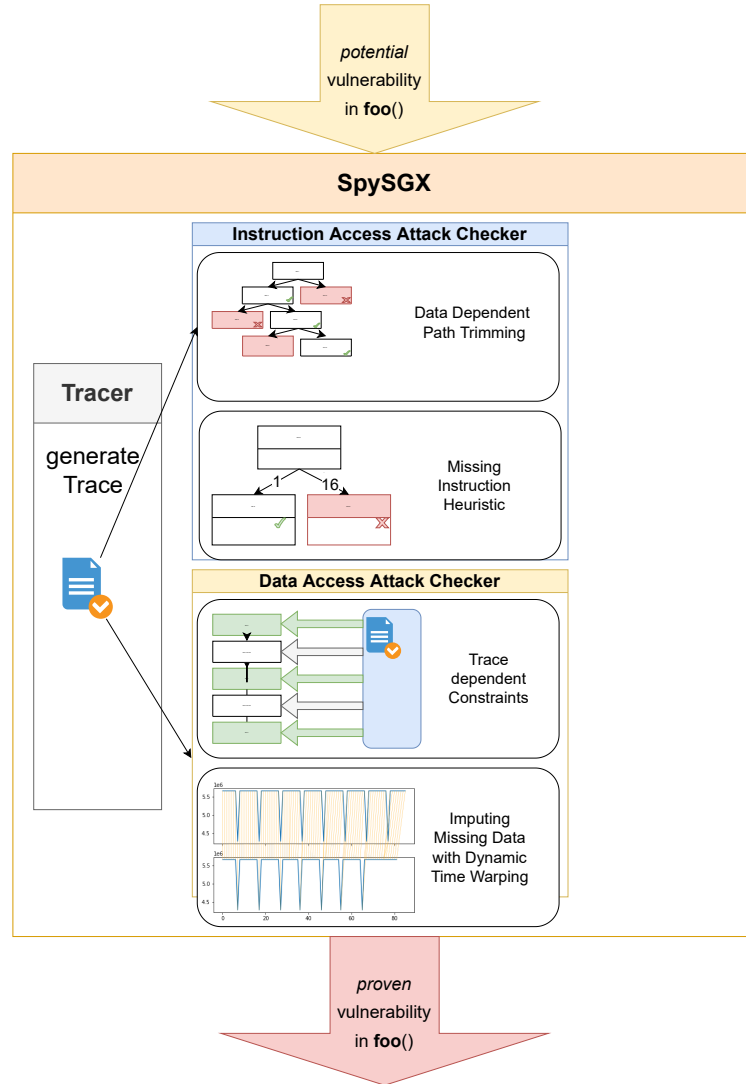


Figure 1.2: Overview of *SpySGX*, our framework to automatically check whether a secret can be leaked from a memory-access trace. *SpySGX* consists of a tracer that collects a memory-access trace and a checker that checks whether the secret can be leaked from the trace. The checker can check either for instruction-access attacks or data-access attacks. For instruction-access attacks, we employ symbolic execution in conjunction with data-dependent path trimming to leak the secret. For data-access attacks, we employ symbolic execution in conjunction with trace-dependent constraints to leak the secret. Each attack also has a strategy to deal with missing data: We develop a missing instruction heuristic for instruction-access attacks to select the most likely branch the enclave took. We use the imputation method: *dynamic time warping* to impute missing data in data-access attacks.

Unlike Faultmorse, our approach makes fewer assumptions about a specific pattern in the data, making it applicable to a broader range of enclaves. Symbolic execution is a technique to execute a program with symbolic values instead of concrete values. *SpySGX* uses a novel approach that combines data science and symbolic execution and can analyze both instruction and data-access attacks. Figure 1.2 shows an overview of *SpySGX*. *SpySGX* consists of two parts: The first part is a tool built on top of the SGX-Emulator Guardian [20] that can collect a memory-access trace of a specific function that we want to analyze. The second part takes the memory-access trace of that function and determines whether the secret can be leaked from the trace.

Path explosion is a problem in symbolic execution where the number of possible paths grows exponentially with the number of branches. Using data-dependent path trimming, we can reduce the number of possible paths to a manageable number. If the enclave does have secret-dependent instruction-accesses, our approach restricts the number of possible secrets to those that could create the specific path. To analyze the effect of missing data on instruction-access attacks, we develop a heuristic to select the most likely branch the enclave took and evaluate the effect of missing data on the attack.

Constraint explosion is a problem in symbolic execution where possible constraints grow exponentially with the number of memory-accesses. Using data analysis to add trace-dependent constraints can simplify the constraints. Here, we stop the symbolic execution engine every time any data is accessed and add a constraint that this access must be equal to the access in the trace. If the enclave has secret-dependent memory-accesses, our approach restricts the number of possible secrets to those that fulfill all constraints. To analyze the effect of missing data on data-access attacks, we apply a modified version of the missing data imputation method *dynamic time warping* [21].

Chapter 2

Background

This chapter introduces various definitions and concepts used in the thesis. In section 2.1, we introduce the Intel Software Guard Extensions (SGX) and describe terminology and concepts used throughout the thesis. In section 2.2, we describe memory-based side-channel attacks and their subcategories instruction and data-access attacks. We also describe different approaches to attack SGX enclaves specifically. In section 2.3, we describe the concept of emulation, a method to help collect memory-access traces in SGX enclaves, and show its advantages and limitations. In section 2.4, we describe the concept of symbolic execution, a method to analyze programs and show their requirements and challenges. In section 2.5, we go over Dynamic Time Warping, a method to compare two time-series with different lengths.

2.1 Intel Software Guard Extensions (SGX)

The Intel Software Guard Extensions (SGX) is a specific instruction set on Intel chips that allow the creation of secure enclaves. These enclaves are designed to provide secure computation on remote computers. From outside the enclave, it is supposed to be impossible to leak data from inside the enclave. Since the architecture of SGX is proprietary, we base our information about SGX on the summary paper *Intel SGX Explained* [1].

One example use case for SGX is to deploy a deep learning model in the cloud [22] while keeping the model parameters secret from the cloud provider, as shown in Figure 2.1. Keeping the model parameters secret from the cloud provider ensures that malicious cloud providers cannot steal the model parameters and that the parameters are safe even if the cloud is hacked. To start, a model owner encrypts and sends their model parameters to the cloud. Inside the cloud, the encrypted model parameters are loaded into the enclave and decrypted. User requests are sent to the

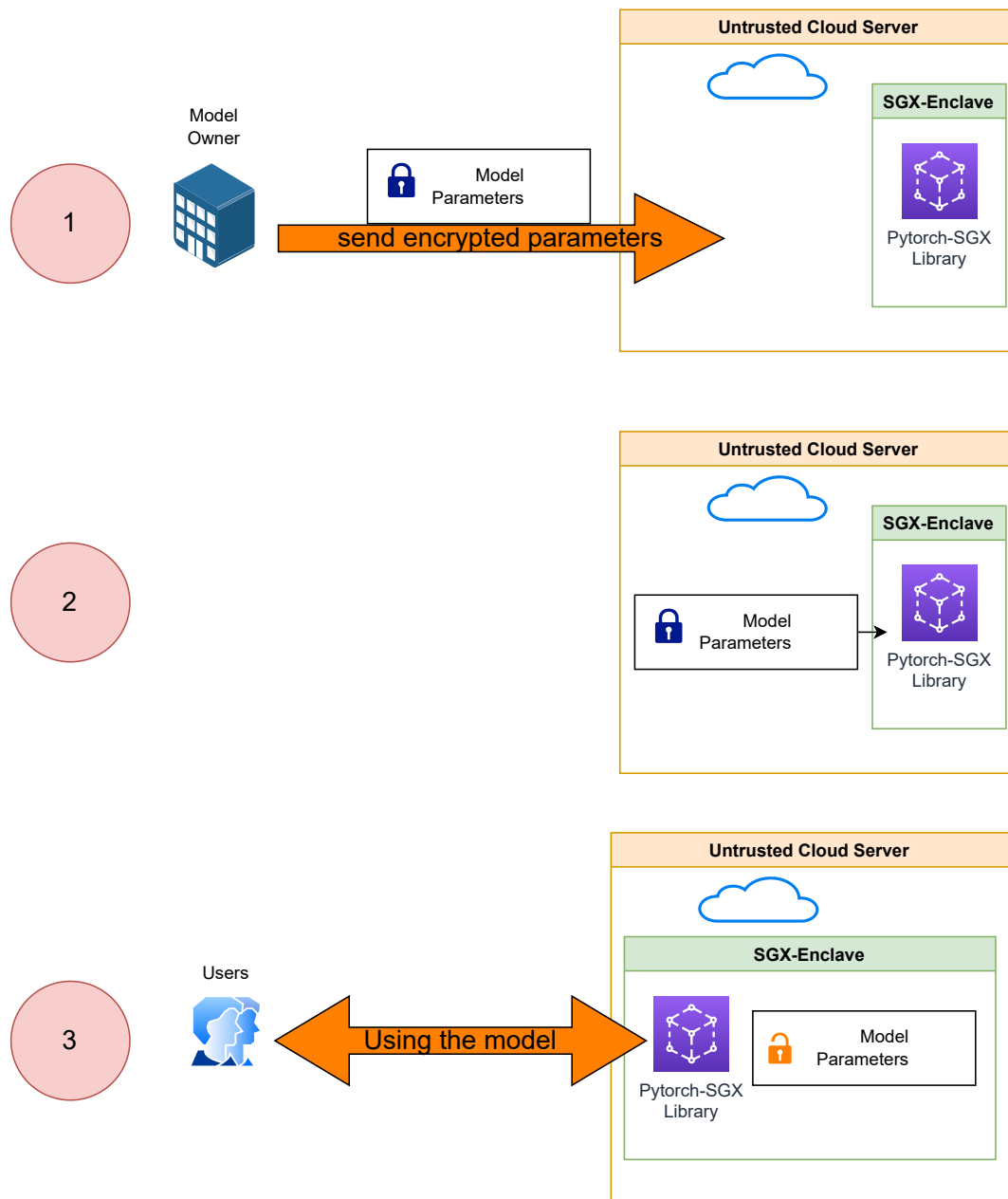


Figure 2.1: Model of an example use case for an SGX enclave to securely deploy deep learning model in the cloud. The model parameters are encrypted in the first step and sent to the cloud provider. Only inside the enclave the model parameters are decrypted and used for computation. Users can send computation requests to the enclave, which will be executed inside the enclave. Since the computation only happens inside the enclave, the cloud provider can not access the model parameters.

enclave, which uses the decrypted model parameters to make predictions. Given that the decrypted model parameters reside within the enclave, the cloud provider cannot access them, keeping them secret.

2.1.1 SGX Terminology and Concepts

The security of SGX relies on memory encryption of all data and code inside the enclave. All enclave memory stays encrypted during all interactions with the enclave, including when the enclave is running.

SGX-enclaves promise data confidentiality; reading enclave data from outside only returns encrypted memory. *Side-channel* attacks can break this confidentiality by making conclusions about unknown information by observing potential side effects like timing differences or power consumption.

Ecalls and *Ocalls* are categories of functions designed to interact with an enclave. An *Ecall* is a function call from outside to inside the enclave, which is necessary to send data to the enclave and call enclave functionality. An *Ocall* is a function call from inside to outside the enclave and is necessary for the enclave to interact with the outside world, for example, to send data back to the user.

Enclave *attestation* is the process of verifying that the enclave is running on a genuine Intel chip and that the enclave has not been tampered with. This process is backed by hardware-based cryptographic keys on the Intel chip. When a user connects to a remote enclave, an *attestation* process occurs, verifying that it is the original enclave and that the expected hardware protections are in place.

An enclave *library* is a program that resides within the enclave. The library provides functions that can be called from outside the enclave to access its functionality. Our example would include a `decrypt` function to decrypt the model parameters and a `predict` function that uses the deep learning model to make predictions given user input.

Enclave *secrets* are confidential values that are supposed to be protected during all interactions with the enclave. While all data inside the enclave is encrypted, enclave secrets are the values the one employing the enclave wants to keep secret. The *pytorch* example enclave secrets include the model parameters and the hardware decryption keys. Enclave secrets depend on the type of enclave library, but cryptographic keys generally count as enclave secrets since they can be used to decrypt other sensitive data.

An *operating system* is software that manages and coordinates all the activities and sharing of resources of a computer. The SGX threat model assumes that an attacker has control over the whole system, including the operating system. Having access to all activities and resources of the computer, an attacker can observe

side-effects of enclave computations, like timing differences or power consumption. These observations can lead to *side-channel* attacks that can break the confidentiality of the enclave.

2.2 Side-channel Attacks on Memory Accesses

A subcategory of side-channel attacks is *memory-access* attacks, which make conclusions from the memory-accesses of an enclave. Side channels are, in essence, the typical data science problem *inference*, where an attacker infers unknown information (secret) from known information (side-effects). A real-life example is observing a hot stove (side-effect), concluding that the stove has been recently used (memory-access), and then inferring that someone has been cooking (secret information).

Memory-access side-channel attacks can be divided into instruction and data-access attacks. In an instruction-access attack, the enclave contains secret-dependent instructions, and an attacker can infer secret information by observing which instructions are executed. A real-life example would be a cooking recipe that states: "If no onions, buy some onions." By observing whether the cook goes to the store, an attacker can infer whether they have onions. In a data-access attack, the enclave contains secret-dependent data-accesses, and an attacker can deduce secret information by observing which data is accessed. A real-life example would be observing which shoe a person buys. By observing the person buying a size 40 shoe, an attacker can infer that the person has a foot size of 25 cm.

2.2.1 Instruction Access Attacks

The instruction-access attack targets secret-dependent control flow, such as if statements. A real-life example can be a cooking recipe that states: If no eggs, buy some eggs. By observing whether a cook goes into a store, an attacker can deduce the unknown information about whether the cook has eggs in storage. In the code example Listing 1, there is an if-condition that is dependent on a secret variable. An attacker can identify whether the secret variable is true or false depending if they observe whether functionA or functionB is accessed.

```
def branchSecret(secret):  
    if (secret == True):  
        functionA()  
    else:  
        functionB()
```

Listing 1: Example of an if-statement that can be attacked with an instruction-access attack.

2.2.2 Data Access Attacks

Data-access attacks attack secret-dependent memory-accesses, like array accesses. A real-life example can be observing which shoe a person buys. By observing the person buying a size 40 shoe, an attacker can infer that the person has a foot size of 25 cm. In the code example Listing 2, an attacker can identify the secret given the linear relation between the secret and the data-access. If `array[0x1000]` is accessed, the secret is 1; if `array[0x2000]` is accessed, the secret is 2, and so on.

```
def accessSecret(secret):  
    result = array[secret * 0x1000]
```

Listing 2: Example of an array access that can be attacked with a data-access attack.

2.2.3 SGX Memory-based Side-Channel Attacks

Many side-channel attacks exist that each observes different side-effects and have different requirements. Attacks can observe page-faults [4], Caches [23], DRAM row buffers [5], branch predictors [24], dependency resolution logic [25] or execution ports [26]. The variety of attacks makes it difficult to mitigate all of them since they target different side effects and have different requirements. Different side-channel attacks have different *granularity* and *noise* and amount of *missing information*. *Granularity* means the accuracy of the information given: A granularity of 4096 bytes means that an attacker can pinpoint memory-accesses within a range of 4096 bytes. Higher granularity means that it is more difficult for an attacker to infer secrets since it is harder to pinpoint where the memory-access happened. *Noise* means the amount of irrelevant information that obscures what the attacker is looking for. *Missing information* means the amount of information the attacker

Table 2.1: Excerpt of various SGX side-channel attacks

Attack vector	Granularity	Noise
DRAM row buffers [5]	1-8 KB	Yes
Dependency resolution [25]	4 B	Yes
Branch prediction [24]	instruction	Yes
controlled-channel [4]	4 KB	No

cannot observe, for example, if there are three memory-accesses in reality, but the attacker observes two.

The CopyCat attack [3] combines *SGX-Step* and the controlled channel attack to leak the entire memory trace of an enclave without any missing values or noise. *Controlled channel* attacks, first demonstrated by Xu et al. [4], can leak the memory-accesses by observing page faults. The attack uses the operating system to regularly unmap the enclave memory. Whenever the enclave tries to access unmapped memory, it triggers a page fault requesting that memory from the operating system. Observing these requests, an attacker generates a trace containing the memory-accesses of the enclave. *SGX-Step* [18] is a framework allowing an attacker to set a timer interrupting the enclave at regular intervals. *SGX-Step* allows the attacker to effectively step through the enclave one instruction at a time, allowing them to take measurements for every memory-access. The attack inferred cryptographic keys from SGX libraries in WolfSSL and Libgcrypt that were previously considered secure.

One limitation of the controlled channel attack is the granularity of 4096 bytes. Different attacks have granularities ranging from 8 kilobytes in the worst case of DRAM row buffers down to 4 bytes for dependency resolution logic. Some attacks like the branch predictor attack [24] also leak only information for instruction-accesses, not data-accesses. These side-channel attacks target shared resources used by the enclave and non-enclave processes. Since non-enclave processes can also access these resources, the memory traces will contain noise unrelated to the enclave memory-accesses. An excerpt comparing various SGX side-channel attacks can be found in Table 2.1.

2.3 Emulation

SGX-enclaves are only available on specific Intel processors. The design also means that memory-access traces are difficult to gather since they are designed to prevent anyone from viewing the memory-accesses. An SGX emulator solves these two

problems; it is software designed to behave like an *SGX* enclave to run enclave libraries inside this emulator. The emulator allows easy tracing of memory-access traces. Emulation is also a requirement for symbolic execution (section 2.4) since they allow replacing absolute values with symbolic values. There exists a wide variety of *SGX* emulators like *OpenSGX* [27], *QEMU* [28], and *Guardian* [20].

Given that emulators are simulated, there are some limitations. Emulators can not emulate hardware-based code like *SGX* attestation: Hardware-based attestation requires hardware-based cryptographic keys, which are not available in the emulator. Some emulators like *Guardian* also take shortcuts while implementing complex *SGX* instructions like *EENTRY* and *EEXIT*. While these shortcuts allow the emulator to run these instructions, side effects might behave differently than in a real *SGX* enclave.

2.4 Symbolic Execution

Symbolic execution is a technique for program analysis using symbolic variables. A symbolic variable is similar to a variable within a mathematical equation. Listing 3 demonstrates a typical use case for symbolic execution: Finding the correct input of a program to reach a specific output.

```
def program(secret):  
    y = secret * 2  
    if y == 12:  
        print("WIN")  
    else:  
        print("LOSE")
```

Listing 3: Example program to analyze with symbolic execution.

The program is first emulated, the variable *secret* is replaced with a symbolic value, and then the program is executed. After symbolic execution, the symbolic calculates two possible paths:

1. $secret \cdot 2 = 12$, output = "WIN"
2. $secret \cdot 2 \neq 12$, output = "LOSE"

The symbolic execution engine then uses a constraint solver to calculate the value of $secret = 6$ that leads to the output WIN.

Symbolic execution faces two challenges: *Path explosion* and *Constraint solving*. Path explosion occurs in symbolic execution because of the vast number of possible execution paths in a program. Each branch in the program’s control flow, such as an if-else statement, leads to a new path. As the number of branches increases, possible paths grow exponentially, leading to a combinatorial explosion of paths that must be explored and analyzed.

Constraint solving poses another significant challenge as the constraints can get arbitrarily complex beyond simple cases like $secret \cdot 2 = 12$. Solving these constraints involves attempting various mathematical simplifications and checking against common cases. If the constraints remain unresolved, the problem is reduced to SAT solving, a Boolean satisfiability problem that can take exponential time to solve. These challenges protect cryptographic algorithms from being broken by symbolic execution.

2.5 Dynamic Time Warping

Dynamic Time Warping (DTW) [29] is an algorithm commonly used to compare two similar time-series that vary in length. Given two time-series, DTW returns a list of tuples that matches the indices of each time-series based on similarity. This list of tuples called *matching*, is calculated via the following simplified steps: At the start of the algorithm, the first index of each time-series is matched. Then one of the three matchings (0,1), (1,1), and (0,1) is selected by checking which has the minimal difference between the two time-series. After selecting a matching, the next three possible matchings are compared until all indices of both time-series are completely matched. In DTW, we can modify a parameter *window size*. This parameter determines the maximal distance two matched indices can be apart.

DTW-based imputation [21] is a method to impute missing data in univariate time-series using DTW. A matching is found by applying DTW on a complete time-series with an incomplete time-series. The imputation happens by replacing missing values with the matched values from the complete trace. The original paper on DTW-based imputation shows that it is more effective at imputing missing data in time-series than traditional methods like spline interpolation.

Chapter 3

Data Description

This section describes how we collect the data and our motivation for choosing the enclave libraries. In section 3.1, we describe our data collection process using emulation to collect memory-access traces. In section 3.2, we describe the two enclave libraries we choose for our analysis. Finally, in section 3.3, we describe the data we collected and how manual analysis can infer the secret from the data.

3.1 Data Collection

For our analysis, we need to collect memory-access traces from enclave libraries. Two possible approaches to collecting memory-access traces are: *hardware-based* and *emulation-based*. We chose an *emulation-based* approach and will explain our reasoning why.

Hardware-based approaches, as used in Faultmorse [17] collect memory-access traces from the physical enclave. Given that *SGX* enclaves are designed to prevent anyone from viewing the memory-accesses, measurements require the use of a side-channel attack to collect the memory-access traces. The advantage of hardware-based approaches is the ability to collect any memory-access traces, including ones that require hardware-specific keys. The disadvantage is that working with side-channel attacks is subject to noise, missing data, and granularity, meaning we do not get the precise memory-access traces.

We choose an emulation-based approach to collect memory-access traces: We use *Guardian* [20], an *SGX*-emulator, to collect the memory-access traces. The reason we choose an emulator is twofold: First, since we are using a simulated environment, we can get accurate memory-access traces without noise. Second, *SGX*-emulation is a requirement for symbolic execution, which we use for the data analysis. *Guardian* is an *SGX* analysis tool is built on top of *angr* [31], a binary analysis framework. We chose *Guardian* because *angr* already implements mem-

ory tracing and symbolic execution.

Our approach has limitations: The *Guardian* emulator has no access to the physical enclave and will act differently than a real enclave. One key security feature of enclaves is attestation, which requires hardware-based cryptographic keys, which are not available in the emulator. Therefore, our approach will not work for attestation. Other limitations are enclave-specific instructions like *EENTRY* and *EEXIT*, which are hardware-based. While these instructions are emulated in *Guardian*, the emulation might behave differently than a real enclave. We have observed that these instructions are called even inside normal enclave functions in OpenSSL [32], so this is a limitation to keep in mind. To limit the impact of these limitations, we only collect memory-access traces from enclave functions that do not contain *EENTRY* and *EEXIT* instructions.

3.2 Target Enclave Library Description

We analyze two categories of memory-access side channels: Instruction-access attacks and data-access attacks. Instruction-access means that the enclave executes an instruction dependent on the secret. Data-access means the enclave accesses a memory location dependent on the secret. While both are memory-accesses and the collection of the data stays the same, the approach to analyzing them is different. We choose *SquareMultiply* as an example of instruction-access attacks and *SubBytes* as an example of data-access attacks.

3.2.1 Square and Multiply

Square and Multiply is an algorithm to calculate the power of a number efficiently and is used, for example, in the Python implementation of *pow()* [33]. A general implementation of Square and Multiply can be seen below.

```
def SquareMultiply(base, exponentbits):
line1  result = 1
line2  for i in exponentbits:
line3      result = square(result)
line4      if i == "1":
line5          result = multiply(result, base)
line6  return result
```

Listing 4: Square and Multiply algorithm

Table 3.1: Line access trace for Square and Multiply with different secret exponents

e	bin(e)	inst1	inst2	inst3	inst4	inst5	inst6	inst7	inst8	inst9
0	0	line1	line2	line3	line4	line6				
1	1	line1	line2	line3	line4	line5	line6			
2	10	line1	line2	line3	line4	line5	line2	line3	line4	line6

The algorithm takes a base b and an exponent e and calculates b^e . Its advantage is its efficiency since its runtime only depends on the number of bits in the exponent e . The downside is that it is vulnerable to side-channel attacks since the If-statement is secret-dependent. An attacker that can observe whether the If-branch is taken can infer the secret exponent.

To demonstrate this in a simplified attack scenario, we assume an attacker knows the sequence of code lines that is accessed. The goal of the attacker is to figure out which e was used. In Table 3.1, we show which lines of code are accessed for different values of e . The attack uses two steps: By observing every "line3" access, the attacker knows that a new loop iteration has started. In the second step, an attacker checks in every loop whether "line5" was accessed, which tells them whether the If-branch was taken. For example, by observing a trace "line3 ... line5 ... line3 ... line3 ... line5", an attacker can deduce that the secret e was "101". Converting this from binary results in the number $e = 5$.

The cryptographic Library *Libgcrypt* v1.6.3 had this Square and Multiply implementation and was shown to be vulnerable to an instruction-access attack [34]. We, therefore, implement a simple SGX enclave that uses this algorithm as an example to test our method of automatically leaking secrets for secret-dependent instruction-accesses.

3.2.2 S-Box Lookup

The Advanced Encryption Standard (*AES*) is the encryption standard for symmetric encryption established by the National Institute of Standards and Technology [35]. Within the AES algorithm, the *SubBytes* function uses an array called S-Box to substitute a byte with another byte. In reality, S-Boxes have been replaced by T-tables [36] for performance reasons, but the principle stays the same, so we demonstrate our method on the S-Box for the sake of simplicity. *AES* contains no If-conditions, preventing instruction-access attacks, but the S-Box is secret-dependent, allowing data-access attacks. MemJam [25] demonstrated such an attack by observing 4-byte granular memory-accesses via a side-channel and recovering the AES key. We, therefore, implement a simple SGX enclave that uses the *SubBytes* algorithm as an

example to test our method of automatically leaking secrets for secret-dependent data-accesses. Listing 5 describes the SubBytes function.

```
def SubBytes(secret):
    result = 0
    for i in range(8):
        currentbyte = secret & 0xff
        result += lookup[currentbyte]
        result *= 0x100
        secret = secret / 0x100
    return result
```

Listing 5: SubBytes algorithm

SubBytes takes a secret integer value between 0 and $2^{64} - 1$ and makes eight memory lookups depending on the secret. We define the address of the *lookup* as *lbase*. For example if our secret value is `0x0102030405060708`, then the program does the lookups `0x08`, `0x07`, `0x06`, `0x05`, `0x04`, `0x03`, `0x02`, `0x01`. We implement this algorithm in an SGX enclave to test automatically leaking secrets for secret-dependent data-accesses.

3.3 Data Overview

In both enclave libraries, the secret value is an integer value that we pass as a function parameter. To collect one memory trace, we run the enclave function with a random secret value and collect either that enclave’s instruction or data-accesses depending on the type of attack. Given that we use an emulation-based approach, our memory traces are byte-granular, meaning we get the exact memory addresses accessed. The number of memory traces we collect depends on the time we have available for the experiment, given that analysis of one memory trace can take up to 20 seconds.

3.3.1 Square and Multiply Data

To analyze the Square and Multiply enclave, we chose 53307 values from a uniformly random distribution between 1 and $2^{31} - 1$ as the secret values. For each secret value, we observe each instruction-access that happens during the execution of the enclave function *SquareMultiply*. Given that we have secret-dependent

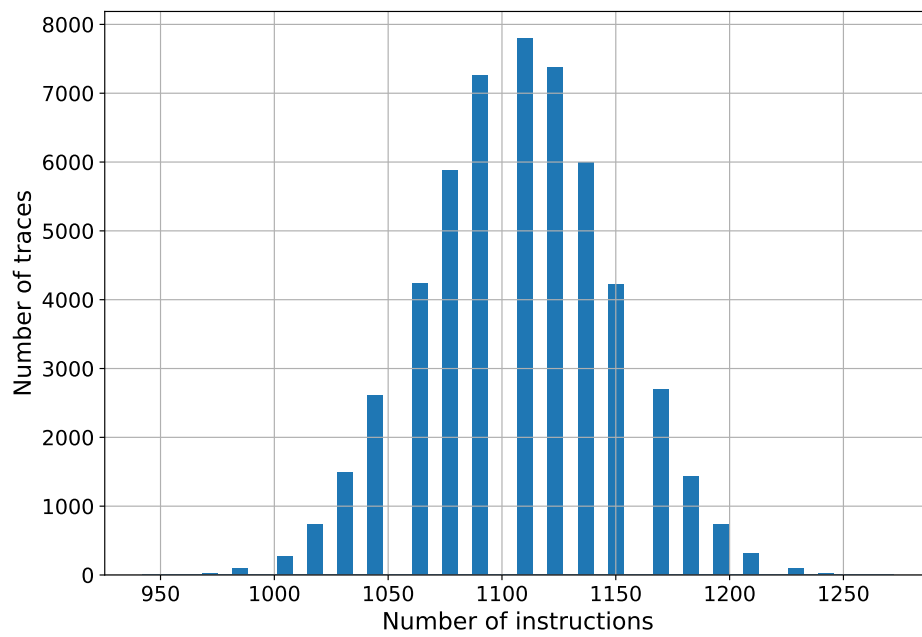


Figure 3.1: Histogram of a number of instructions in the Square and Multiply enclave. On the x-axis, we have the number of instructions; on the y-axis, we have the number of memory traces with that number of instructions. The number of instructions ranges from 942 to 1272 and seems normally distributed, with a mean of 1107.21 and a standard deviation of 40.92.



Figure 3.2: Instruction-access trace of a simplified SquareMultiply enclave. In the simplified enclave implementation, the square and multiply loop is run 8 times, once for each bit in the secret exponent. The trace contains all 284 instruction addresses that were accessed during the execution of the enclave with the secret exponent 42. In each loop iteration, the square function is called, which is located at address 0x4000. If the current bit of the secret is 1, the multiply function is called, located at address 0x5000. We can see the beginning of each loop in the graphic, where the trace reaches 0x4000. The binary representation of 42 is 00101010, which has a 1-bit in the third, fifth, and seventh index. This is represented in the trace, where we can see that the multiply function is called in the third, fifth, and seventh loop iterations.

secret	mem0	...	mem7	mem8	...	mem17	...
0x010203040506 0708	5670648	...	lbase + 8	5670668	...	lbase + 7	...
0x010203040506 0103	5670648	...	lbase + 3	5670668	...	lbase + 1	...

Table 3.2: First two rows of the memory trace for the S-Box Lookup enclave, where lbase is the base address of the S-Box. The secret is an 8-byte key value and 8 memory-accesses are linearly correlated with the secret. We can see that the memory-accesses mem7 and mem17 are linearly correlated with the secret.

instruction-accesses, we expect to see a difference in the number of instruction-accesses for each secret value. In Figure 3.1, we show the distribution of the number of instructions for all 53307 memory traces. The length of each memory trace ranges from 942 to 1272 instruction-accesses and seems normally distributed with a mean of 1107.21 and a standard deviation of 40.92. We store the data with the secret value as the first column and the instruction-accesses as the remaining columns. Given that some traces are longer, we pad shorter traces with NaN values to fill out all 1272 columns. The values of each instruction-access range from 4206592 to 4214926, representing the memory addresses of the instructions. These addresses have a mean value of 4212451.27 and a standard deviation of 3474.81. In Figure 3.2, we can see what the last 284 instruction-accesses look like for the secret value 42.

3.3.2 S-Box Lookup Data

To analyze the S-Box Lookup enclave, we select 60000 uniformly random secret keys between 0 and $2^{64} - 1$. For each secret value, we observe each data-access during the execution of the enclave function *SubBytes*. During each program execution, we get exactly 86 data-accesses, which we call mem0 - mem85.

As we can see in Table 3.2, **mem7** and **mem17** are both linearly correlated with the secret. We know 8 of those data-accesses are correlated with the secret, and all other accesses are unrelated. The challenge is to find those columns and build a predictor that can predict the secret value given a list of memory-access mem0 - mem85. Mem0 to mem85 are valued between 4276256 and 5670680 with a mean of 5540961.08 and a standard deviation of 404988.88.

3.3.3 Missing Data

In our analysis, we also consider the case where an attacker does not get the complete memory trace which can happen for multiple reasons. One reason can be that the attacker chooses a specific side-channel attack like the stealthy page table attack [9] that misses memory-access. The other reason could be mitigations like

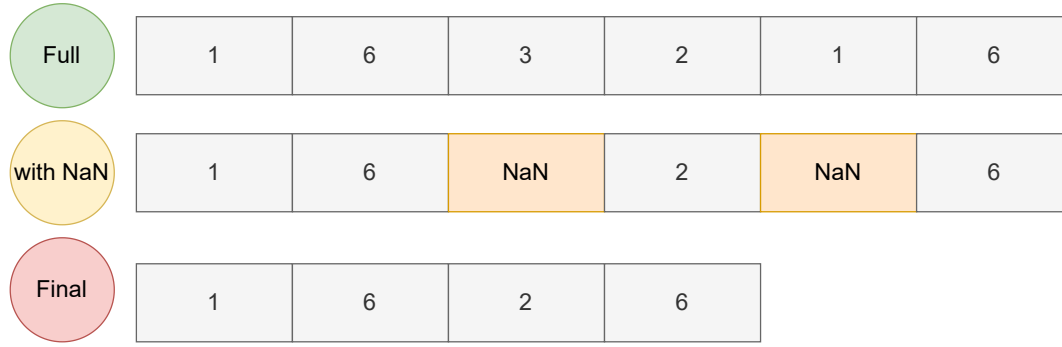


Figure 3.3: Our method of generating traces with missing data. We first take a complete trace and then apply a missing factor p to each element in the trace. We then remove all NaN values from the trace and get a trace with missing data where it is unknown which elements are missing.

AEX-Notify [19] that make single-stepping [18] infeasible. To test the effectiveness of *SpySGX* in these conditions, we make the simplified assumption that each element in the trace has an equal probability of being lost. To create traces with missing data, we define a missing factor p as the probability of an element in the trace disappearing. We then apply the missing factor to our data set and create a new one with missing data. As seen in Figure 3.3, commonly missing data are generally marked with NaN values. However, since an attacker would not know which accesses are missing, we remove the NaN values from the data set.

For the Square and Multiply enclave, we create a new data set with missing data for each missing factor p between 0.05 and 0.95 with a step size of 0.05. We chose this step size, which allows us to analyze the changes between missing factors while still having enough traces for each missing factor: This split leaves us with 19 data sets of around 2805 traces each. This number is not exact since we stopped the experiments in the middle due to time constraints. For the S-Box Lookup enclave, we create a new data set with missing data for each missing factor p between 0.05 and 0.50 with a step size of 0.05. We only analyzed up to a missing factor of 0.50 since the number of redundant data-accesses in data-access attacks is low. Higher missing values would change the attack scenario more akin to a brute-force attack which is not the focus of this work. This split leaves us with 10 data sets of 6000 traces each.

Chapter 4

Methodology

To tackle the inference problem of whether we can recover the secret from the memory trace, we combine data analysis with symbolic execution.

We have an enclave that uses a secret-dependent code path and an enclave that uses secret-dependent data-access. While *SpySGX* always uses *symbolic execution* to recover the secret, we use different approaches for the two enclaves. In the case of secret-dependent code paths, we use the trace to trim the symbolic execution tree to address the path explosion problem. For secret-dependent data-accesses, we use the trace to add additional constraints to the symbolic execution engine. Only these additional constraints allow us to recover the secret.

We first test the effectiveness of the approaches on the two enclaves with full traces by using them to recover the secret and comparing the results to the original secret. Afterward, we test the approaches on partial traces to see how much of the trace is needed to recover the secret.

4.1 Symbolic Instruction-Access

During normal symbolic execution, the symbolic execution engine explores all possible paths through the program. It does so by tracking the program's current state containing the memory and register values. The engine creates a new state for each possible branch. This process continues until the engine has found a state that matches the required outcome. The symbolic execution engine can then analyze this state to generate an input that leads to this state.

For our *SquareMultiply* enclave, the standard approach has the problem of state explosion. A secret exponent of 31 bits leads to a state tree of 2^{31} states. The number of states grows exponentially with the secret exponent size. State explosion is a general problem in symbolic execution, preventing the analysis of cryptographic algorithms.

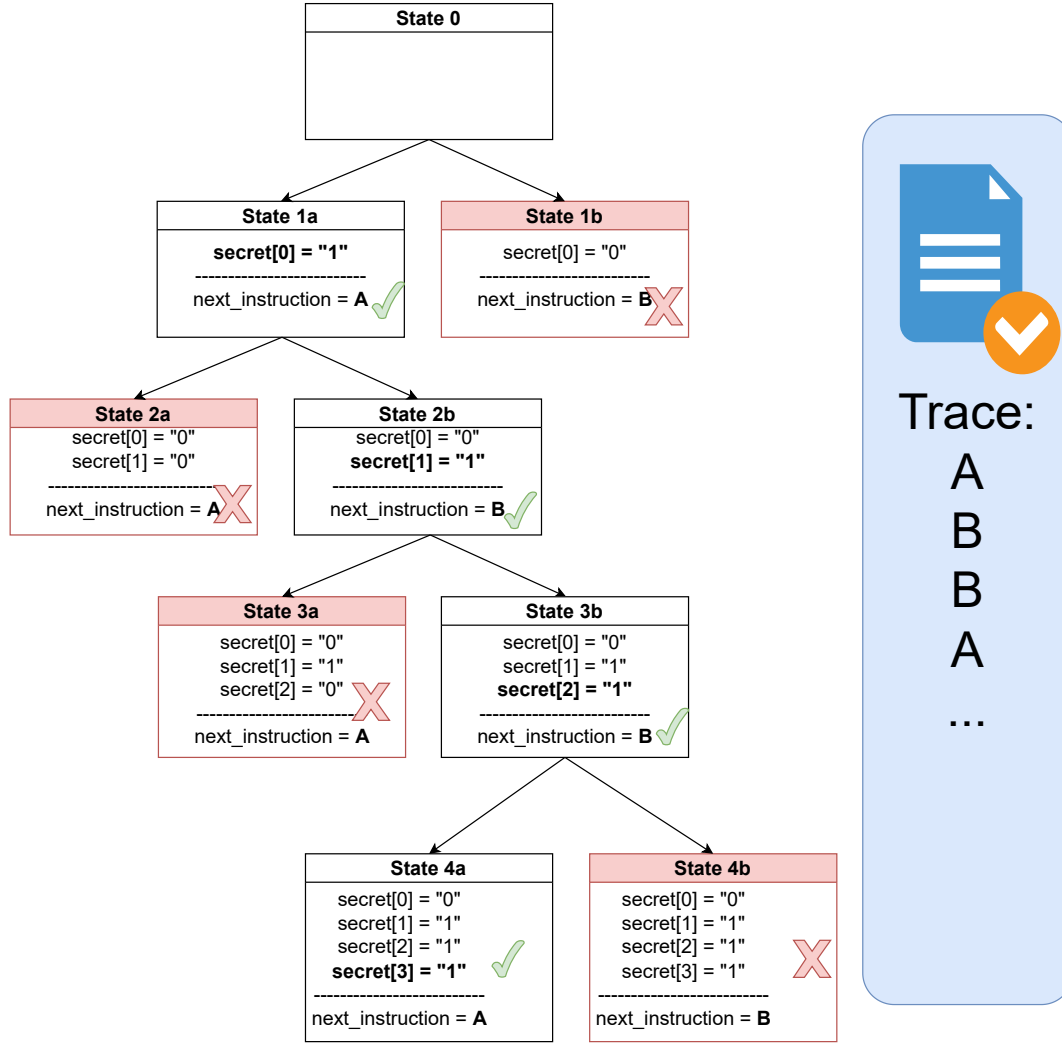


Figure 4.1: Demonstration of our approach to trim the symbolic execution state tree. We check whether the next instruction matches the instruction trace at each branch. We can discard the branch if the instruction does not match the trace.

We can trim the state tree with the instruction trace, tackling the state explosion problem. In our novel approach, we keep track of our current position in the instruction trace as seen in Figure 4.1. At each branch, we can check whether the next instruction matches the instruction trace and discard all states that do not match.

4.2 Symbolic Data Access

The *SubBytes* enclave has no secret-dependent code path. While we know the secret linearly depends on 8 of the 86 data-accesses, we want a generic approach that makes only the assumption that the secret is somehow correlated with the data-accesses. Therefore we tweak the symbolic execution engine to add a condi-

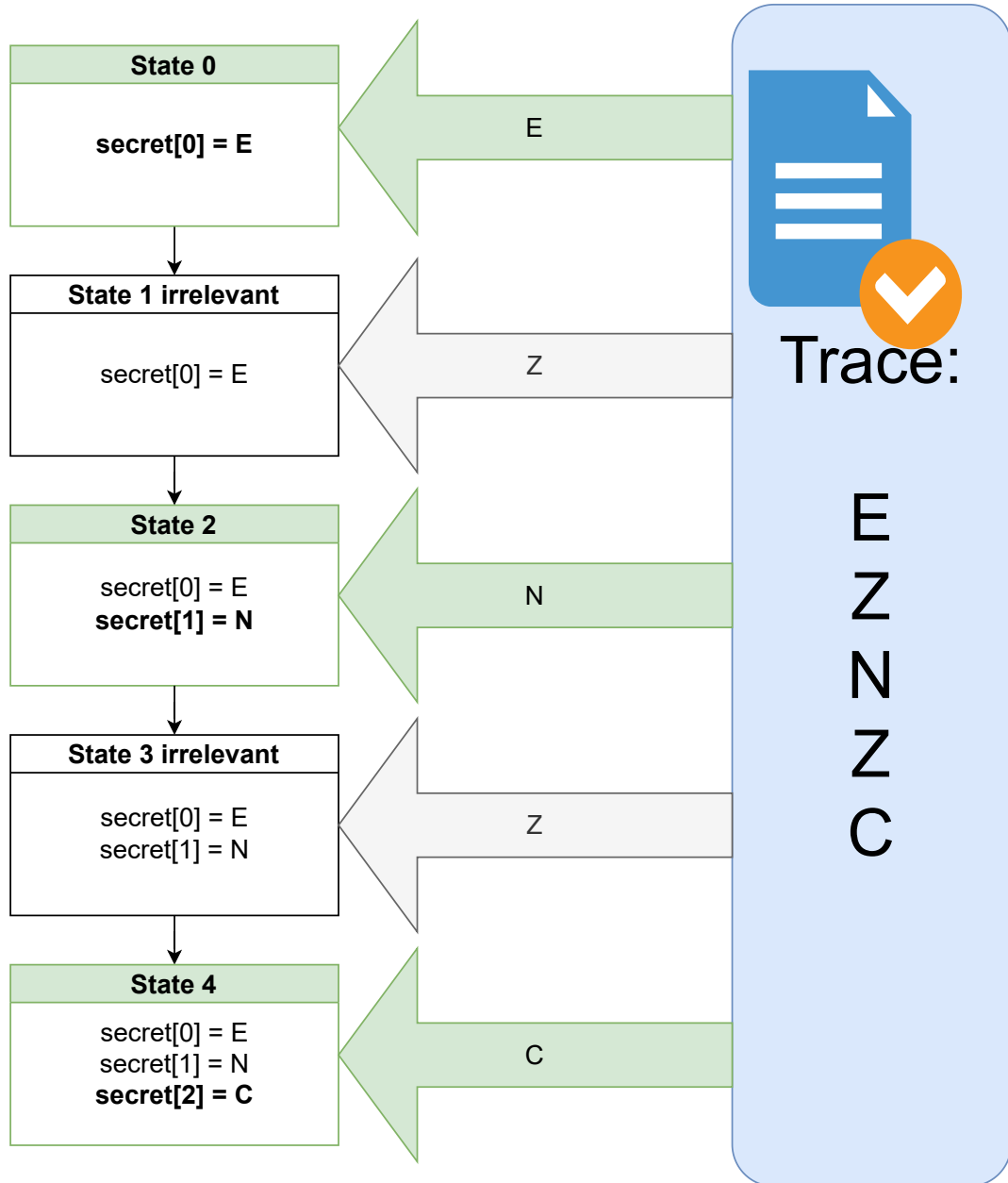


Figure 4.2: Demonstration of our approach to deal with secret-dependent data-accesses. We add that the access must match the trace at each data-access. While adding a condition for each irrelevant static data-access, we can still recover the secret since these conditions are always true.

tion to each data-access. The condition is that data-accesses must match the trace shown in Figure 4.2. While this approach adds a condition for each irrelevant static data-access, we can still recover the secret since these conditions are always true. Ultimately, we get a symbolic expression that we can solve to recover the secret.

4.3 Dealing With Missing Data

In contrast to typical missing values, we do not know where the data is missing, which prevents us from using interpolation approaches like Linear Interpolation, Spline Interpolation, and Nearest Neighbour Interpolation. For instruction traces, we have the advantage of multiple instructions per branch. To use this, we develop a missing data heuristic for instruction traces to decide the most likely code path given the available trace. In contrast, in our example of the data-access enclave, we depend on each secret-dependent data-access. To address this problem, we reconstruct the original trace with a modified dynamic-time-warping-based imputation algorithm to recover the secret.

4.3.1 Missing data heuristic

We can recover the secret in our *SquareMultiply* enclave as long as we can correctly infer all the branches the enclave has taken. Each branch consists of multiple instructions unique to that branch, which means that as long as our trace did not delete all instructions unique to that branch, we can infer the secret. An example can be seen in Figure 4.3 where the original trace was "Inst1, Inst2, Inst3", but "Inst2" is missing. "Inst1" and "Inst3" still uniquely identify the left branch, allowing us to discard the right branch.

To make this analysis automatically, we define the *Missing data Heuristic*: The heuristic is based on the assumption that the path with fewer missing instructions is more likely to be the correct path. Our heuristic performs the instructions individually and checks whether the instruction is available in the trace. Instructions that are not in the trace are counted as missing instructions. At each branching path, we instruct the symbolic execution engine (Figure 4.3) to take the branch with the fewest missing instructions and discard the other branches. In the case of a tie, both branches are kept until there is a point when one has fewer missing instructions.

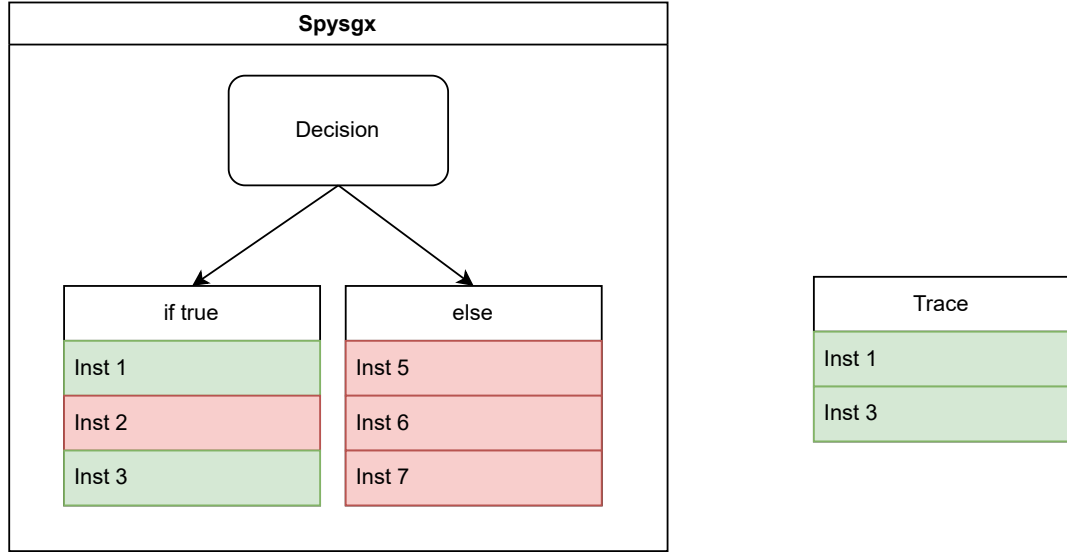


Figure 4.3: How the Missing Instruction Heuristic chooses the correct branch: The original trace was "Inst1, Inst2, Inst3", but "Inst2" was lost. We have two possible states containing the addresses of the instructions that it would execute. Our heuristic performs the instructions individually and checks whether the instruction is available in the trace. Instructions that are not in the trace are counted as missing instructions. In our case, the left branch has only one missing instruction: Inst2, while the right branch has three missing instructions. Therefore the right branch is discarded, and we continue our analysis.

4.3.2 Imputing missing data with dynamic time warping

For our *SubBytes* enclave, we know that the secret is linearly dependent on 8 of the 86 data-accesses. We can recover the secret if the linearly dependent data-accesses are not missing. The challenge is to impute the missing data at the correct indices.

To impute the missing data, we use a Dynamic Time Warping (DTW)-based approach that imputes missing data. In Figure 4.4, we can see how DTW works with different window sizes. Given two time-series, DTW finds the best matching between the two time-series. Given the matching, we calculate the missing indices by finding matchings where multiples of the complete trace match one index of the incomplete trace. We choose the corresponding values of the complete trace to impute the missing values. We attempt the analysis with different window sizes to find the best window size for imputation. We provide a more detailed description of DTW in section 2.5 for the interested reader.

4.4 Experimental Setup

We expect our approach to recover the secret every time if the trace is complete. To verify this claim we randomly select 1000 traces from our Square and Multiply

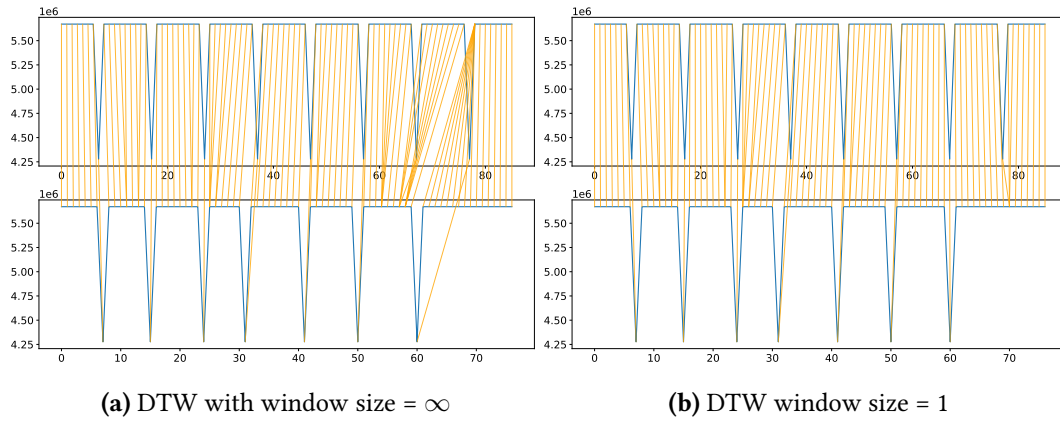


Figure 4.4: Demonstration of the disadvantage of large window size in Dynamic Time Warping (DTW). In the context of DTW, window size determines the maximal distance two matched indices can be apart. The task is to match a complete data-access trace (top) with an incomplete data-access trace (bottom). The incomplete trace is missing some values, including the last secret dependent access. The yellow lines represent the matching made by DTW. In the left image, we see a DTW matching with window size = ∞ : To accommodate the last missing trace, DTW significantly shifts the indices. This shift has undesirable results, like secret-dependent access 2 in the incomplete trace gets matched with secret-dependent access 3 in the complete trace. The right image shows a DTW matching with window size = 1. In contrast to the first matching, this matching correctly matches the secret-dependent accesses since it cannot make significant shifts.

enclave data (subsection 3.3.1) and 1000 traces from our S-Box Lookup data (subsection 3.3.2). To evaluate the effectiveness of our approaches on full traces, we randomly select 1000 traces from each enclave and compare the reversed secret with the original secret. A secret is reversed correctly if the original and the reversed secret are equal. To evaluate our method of dealing with missing data, we randomly select 50000 traces as a base dataset. Given that both enclaves contain different amounts of information, we use two different approaches. Thus, we split the evaluation into missing instructions access and missing data-accesses. We define a missing factor p that determines the likelihood of an instruction or data-access being missing.

4.4.1 Missing Instructions-Accesses

In the instruction traces of the *SquareMultiply* enclave, we are interested in how little of the trace is needed to recover the secret. We split our 50000 traces into buckets of 2500 traces with a missing factor p ranging from 0.05 to 0.95. We use *SpySGX* to reverse the secret and compare it to the original secret. We distinguish between 4 results:

1. *Correct* The secret was reversed correctly.
2. *Bug* *SpySGX* aborted with an error.
3. *State Explosion* More than 256 states are active. Aborted.
4. *Wrong* The secret was reversed incorrectly.

For *State Explosion*, we had to choose an arbitrary threshold to abort the analysis since at some point; we can assume that the analysis will not finish due to state explosion. We chose 256 since a similar threshold is used in Guardian [20] for their symbolic execution analysis of memory corruption vulnerabilities in SGXenclaves.

4.4.2 Missing Data-Accesses

In the data-access traces of the *SubBytes* enclave, we deal with the issue that a secret is irreversible if we lose one of the 8 relevant data-accesses. On the other hand, the window size of DTW is only relevant if we lose one of the relevant data-accesses. We analyze the missing factors $p = 0.1 - 0.5$ in steps of 0.1. For each missing factor, we analyze the accuracy of our imputation method by comparing the imputed trace with the original trace. We use the following window sizes for DTW: 1, 0, ∞ , *dynamic*, where the window size is the number of missing data points. We expect the window size of 1 or dynamic to be the most accurate since it

limits the matching to the closest indices. We also compare the accuracy of perfect detection of the missing indices. We analyze the accuracy of recovering at least 7 out of 8 relevant data-accesses.

Chapter 5

Results

To evaluate whether the secret can be recovered from the memory trace, we combine data analysis with symbolic execution. For our evaluation, we use the two victim enclaves as described in section 3.2. One with a secret-dependent code path and one with a secret-dependent data path. While SpySGX always uses symbolic execution to recover the secret for both enclaves, the execution trace of secret-dependent code path is used to trim the symbolic execution tree to address the path explosion problem. For the secret-dependent data-accesses, SpySGX adds additional constraints to the symbolic execution engine. With those optimizations, we can recover the secret with reasonable time and memory usage.

In this chapter, we present the results of our evaluation of *SpySGX* on full traces and our evaluation of our method of dealing with missing data. In section 5.1, we present the results of the *Square and Multiply* enclave and put them into context with other approaches. In section 5.2, we present the results of the *SubBytes* enclave and the issues we faces regarding constraint explosion. Finally, in section 5.3, we present the results of our method of dealing with missing data.

5.1 Symbolic Instruction Access

In the case of the *Square and Multiply* enclave, *SpySGX* was able to reverse the secret in all 1000 test cases. To put our results in perspective in the case of the *Square and Multiply* enclave, we compare the runtime with other approaches in Figure 5.1. We provide a prime p with n bits and ask the approaches to solve for the secret exponent d given $b^d \bmod p$. The prime p is chosen to be not smooth, i.e. $p - 1$ is not factorizable into small primes to prevent easy solutions via the Pohlig-Hellman algorithm [37]. The first approach is unmodified symbolic execution, which attempts to reverse the secret exponent by checking all paths until one is found that leads to the correct result. The second approach is brute forcing, which tries all possible

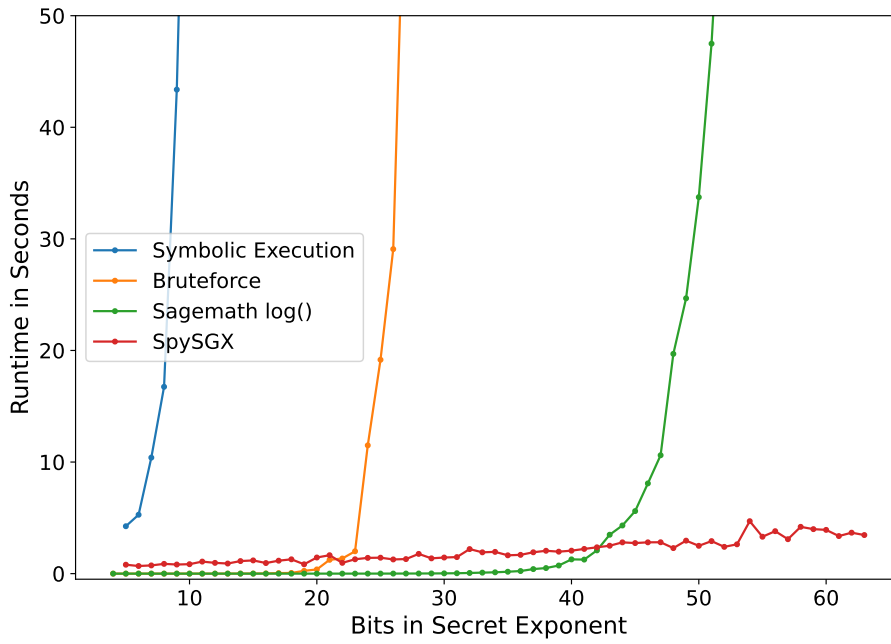


Figure 5.1: Runtime comparison of *SpySGX* with other approaches on the *Square* and *Multiply* enclave. We ask the algorithms to solve for the secret exponent while providing the number of bits of the secret exponent. The algorithms tested are unmodified *Symbolic Execution*, *Bruteforcing*, the *log* algorithm in *SageMath*, and *SpySGX*. Since runtimes can grow exponentially, each measurement was only performed once due to time constraints. Due to the implementation details of the enclave, we could only test up to 64 bits. We can see that all approaches except *SpySGX* grow exponentially with the number of bits and are not feasible for secrets with at least 64 bits.

values for the secret exponent until the correct value is found. The third approach is the *log* algorithm in *SageMath*, which uses specialized algorithms to solve this problem. The last approach is *SpySGX*. We can see that all approaches except *SpySGX* grow exponentially with the number of bits and are not feasible for large secrets. *Square and Multiply* can be used in cryptographic algorithms like Diffie-Hellman, where the secret exponent is recommended to be 2048 bits or larger [38]. Given that best of the approaches *SageMath* already struggles at reversing 50 bits, we can see that *SpySGX* is the only feasible approach for big secrets if a memory trace is available.

5.2 Symbolic Data Access

In the case of the *SubBytes* enclave, *SpySGX* was able to reverse the secret in all 1000 test cases for complete traces. While reversing the secret in the data-access trace, we noticed surprisingly high algorithm runtimes of 20 seconds per trace, which is 7 times slower than in the instruction-access enclave. This difference is surprising because the data-access enclave is significantly smaller than the instruction-access enclave. We assumed this slowdown is caused by the high number of constraints generated by the symbolic execution engine. Therefore, we tested reversing the secret on *tinyAES*, a tiny implementation of AES [39]. We noticed high memory usage of 4GB and a complete halt of progress after just 5 % of the program execution. Reaching this point took 20 seconds; we waited 5 minutes, but the program did not progress, so we terminated the program. These issues were caused by the high number of constraints generated by the symbolic execution engine. In theory, having the additional information through the memory trace should reduce the number of constraints. We have two hypotheses why this is not the case: One explanation could be that our implementation does not force the symbolic execution engine to simplify the constraints. Due to time constraints, we could not investigate whether this was the case. In this case, our approach still works but requires tweaks in the implementation. The other explanation is that solving constraints is an NP-complete problem that, in the worst case, takes exponential time to solve[40]. In that case, our data-access approach does not generalize to complex cryptographic enclave Libraries since we have to deal with exponential constraint growth.

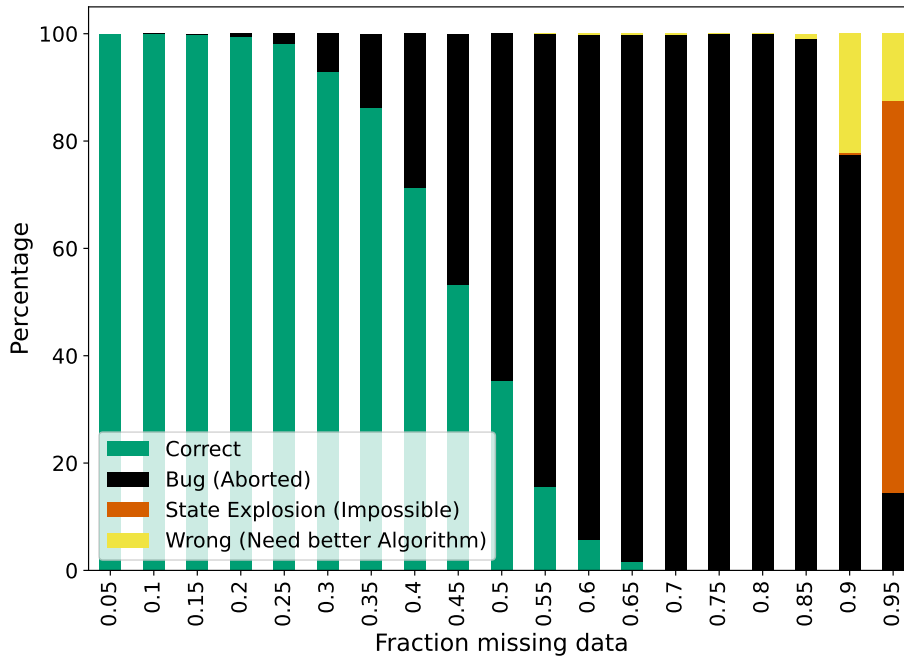


Figure 5.2: Effectiveness of our heuristic at reversing the secret from an incomplete trace. Each bar represents 2500 traces. The x-axis determines the percentage of the missing data. The different colored bars determine the result of the Heuristic. **Correct** means the secret was reverted correctly. **Bug** means that a bug in the implementation aborted the current execution. **State Explosion** means we reached the threshold of 256 states and assume we are in a state explosion. **Wrong** means that after reversing it returned the wrong secret

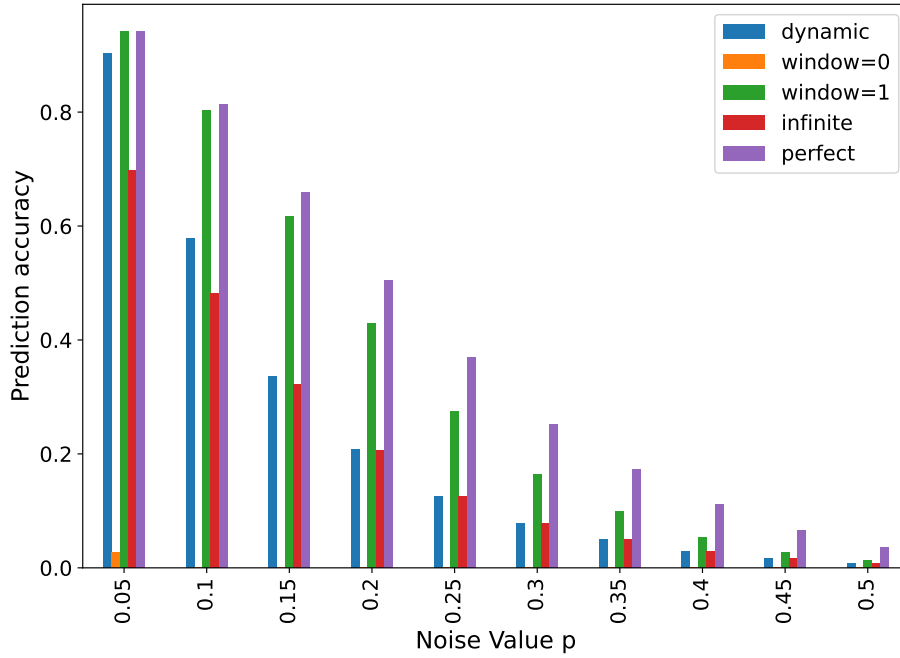


Figure 5.3: Effectiveness of Dynamic Time Warping at imputing missing data. The x-axis determines the percentage of the missing data. On the y-axis is the percentage of traces where at least 7 out of 8 bytes were correctly reversed. Each bar represents a different window size. **dynamic** is a window size equal to the number of missing values. **infinite** is a window size equal to the complete trace’s length. **perfect** is a reference value that assumes perfect imputation of the missing data. When reversing at least 7 out of 8 bytes, a window size of 1 performs the best.

5.3 Dealing With Missing Data

To analyze how much of the trace is needed to leak the secret, we also demonstrate two approaches to deal with incomplete traces. In our first approach, we are attacking an enclave with redundant information, meaning that leaking the secret is not dependent on one single memory-access. Here, we use a missing instruction heuristic to determine the correct branch for the symbolic execution. The results of the Missing instruction Heuristic can be found in Figure 5.2. It can leak the complete secret with 98 % accuracy with 25 % of the trace missing. Afterward, the prediction accuracy decreases to less than 1 % at a missing factor $p = 0.7$. We also see a high number of *Bug* results, which shows a range to improve and that our approach might be able to reverse the secret at higher missing factors up to $p = 0.8$. Our approach effectively reverses the secret from an incomplete trace on the *SquareMultiply* enclave.

With a second approach, we attack an enclave with non-redundant informa-

tion, meaning that leaking the secret is dependent on one single memory-access. To accurately pinpoint the correct memory-access, we use dynamic time warping to impute missing data [21]. The results of our DTW-based approach for imputing data can be found in Figure 5.3. We test different window size parameters for dynamic time warping and find that a window size of 1 is optimal. We show that our imputation method is within 10 % of a theoretical perfect imputation method. We analyzed the effect of different window sizes on the accuracy of our predictions. On reversing the entire secret, different window sizes all perform the same as perfect imputation. This result is expected since the window size is only relevant if we lose one of the relevant data-accesses. On reversing at least $7/8$ bytes of the secret, we see that a window size of 1 performs best but is not optimal compared to perfect imputation.

Chapter 6

Discussion

In this chapter, we discuss the implications of our results and how they can be used to improve the security of *SGX* enclaves. We also discuss the limitations of our approach and how they can be addressed in future work. We conclude this chapter by describing related work and how our approach differentiates itself from previous work.

6.1 Implications

Memory-based side-channel attacks are a significant threat to the confidentiality of *SGX* enclaves, which attacks on the official GNU cryptographic library Libgcrypt [9] and the FIPS-certified cryptographic library WolfSSL [3] demonstrate. By using data analysis to limit the branches of symbolic execution and adding additional constraints, we tackled the path-explosion problem that prevented unmodified symbolic execution from analyzing memory-based side-channel attacks on *SGX* enclaves. Our approach has the advantage that it is fully automated and does not depend on specific patterns that emerge from the memory trace. Using our novel approach, developers can verify potential memory-based side-channel attacks on *SGX* enclaves and focus on fixing these instead of false positives. This will save valuable time and resources for developers and improve the security of *SGX* enclaves.

Our approach combining data science and symbolic execution can also be used in other fields, potentially providing new insights. While our work focused on memory-based side-channel attacks *SGX* enclaves, our approach can also be applied to generic memory-based side-channel attacks since our approach is not dependent on the enclave’s architecture or operating system: *SpySGX* is built on top of Guardian [20], which is built on top of the symbolic execution engine *angr* [31], which supports a wide range of architectures and operating systems.

Our work also adds to the discussion of the effectiveness of mitigations like AEX-Notify [19] that limit the number of memory-accesses an attacker can observe. We show that our approach can reverse the secret even from an incomplete trace if not too many memory-accesses are missing. An interesting observation is that instruction-access traces are more robust to missing data than data-access traces because the instruction-access traces contain more redundant information than data-access traces. Based on our results from chapter 5, the AEX-Notify mitigation would need to limit the number of memory-accesses to less than 35 % in case of a similar implementation of the *SquareMultiply* enclave. Worth noting is that our approach is not optimal, and around 50 % of the results are *Bug* results, and only at 95 % missing traces can we guarantee that the secret is not leaked. We expect the *Bug* results to be easier to improve since it only requires a bug fix. *Wrong* results can be eliminated by using a better heuristic. We expect that most *State Explosion* are hard to solve since they imply that there were too many ties in the heuristic and the trace is too incomplete to recover information. Given that an attacker can take multiple measurements to reduce the number of missing traces, this raises the question of how effective the AEX-Notify mitigation is in practice.

6.2 Limitations and Further Work

Given the complexity of SGX enclaves, our approach has some limitations. One limitation of our work is that we only tested our approaches on two sample enclaves. While our approach is generalizable to other enclaves, future work should verify this assumption on real-world enclaves. One concern of ours is the high memory consumption of our approach regarding analyzing data-access attacks. In theory, the symbolic constraints should cancel each other out, simplifying the constraints and reducing memory consumption. However, in practice, the constraint solver might not be able to simplify the constraints leading to the exponential growth of the constraints and memory consumption. Another concern is enclave functions that use hardware-specific instructions like *EENTER* and *EEXIT* instructions. *Guardian* simulates these instructions, but it is unclear how this affects the symbolic execution and whether this affects the effectiveness of our approach. Future work should analyze the effectiveness of our approach on real-world enclaves. Good targets would include older versions of the Libgcrypt enclave, which is known to be vulnerable to memory-based side-channel attacks [17].

Our approach is also limited to functions that we can symbolically execute. This limitation means that enclave functionality like attestation requiring hardware features that cannot be emulated is not supported. While this is a limitation that our

emulation-based approach suffers, we believe that the advantages of our approach outweigh this limitation. Using emulation guarantees that our approach is not dependent on specific hardware to analyze enclaves. It also allows us to generate traces with perfect accuracy, which is not as simple in hardware-based tracing. It makes our approach more accessible to researchers and developers who do not have access to the hardware that supports SGX enclaves.

Lastly, we assume the missing data is randomly distributed, which does not accurately reflect the real world. For example, in the case of an attack that observes instruction-accesses, an attacker is more likely to observe instructions with longer execution times [9]. Given the plethora of possible attacks, we deemed it impractical to analyze the effect of missing data on all possible attacks and opted to use randomly distributed missing data. Whether this affects the effectiveness of our approach is an open question, but we believe it does not significantly affect our results. For future work to better understand the effectiveness of the AEX-Notify mitigation, they would need to analyze the effect of missing data on real-world enclaves with a more optimal approach to dealing with missing data.

6.3 Related Work

Automated verification of memory-based attacks is a recent field of research. Faultmorse [17] was published in 2023, the same year as our work, and is, to our knowledge, the only other work towards automated verification to detect memory-based side-channel attacks. Faultmorse uses the controlled-channel attack [4] to collect page fault sequences from enclaves. Page fault sequences are memory-access patterns with the limitation that the last 12 bits of the memory addresses are unknown. Inside these page fault sequences, they identify the longest recurring sequence. By counting the number of instructions between these sequences, they identify between 1-bits and 0-bits of the secret and recover it. Our approach is more generic and can be applied to more enclave implementations. Using emulation, we can accurately collect the memory-access traces, allowing us to analyze more fine-grained attacks where an attacker can observe more granular memory-access patterns than page faults. Symbolic execution also allows us to analyze data-access and instruction-access attacks, while Faultmorse is limited to instruction-access attacks. Regarding instruction-access attacks, our approach does not require specific repeating patterns to leak the secret-dependent instructions, making it applicable to more enclaves. Lastly, we showcase methods to deal with missing data, which is not addressed in Faultmorse.

In our work, we use the Guardian [20] framework to emulate SGX enclaves and

collect memory traces. Guardians is also a framework for the automated detection of vulnerabilities in *SGX* enclaves using symbolic execution. Guardian and its predecessor TEE-REX [41] analyze another type of vulnerability in *SGX* enclaves: memory corruption vulnerabilities. While these attacks are also memory-based, they are not side-channel and require entirely different approaches.

The ENCIDER [16] framework uses symbolic execution to detect Timing and Cache side-channels automatically. The framework builds an internal representation of timing and cache behaviors. It uses symbolic execution to detect whether secrets lead to different execution times or cache behavior. While ENCIDER is a powerful tool to detect side-channel vulnerabilities, it does not provide a way to recover the secret. Therefore, we believe our work complements ENCIDER: ENCIDER can be used to detect potential side-channel vulnerabilities, and our work can be used to verify that the enclave is indeed vulnerable to a side-channel attack.

Another approach to analyzing side-channel attacks is the use of Machine Learning. Deep learning has also been applied to power side channels [42], recovering a full AES key after 1300 traces. While this approach is promising, it requires many traces to recover the secret. Furthermore, it is unclear how well this approach generalizes to other enclaves and the memory-based attack. As long as deep learning has not been applied to memory-based side-channel attacks, we cannot compare our approach to this approach.

On the topic of imputing missing data in memory traces, the stealthy Page Table [9] attack used attack-specific regular expressions to detect the indexes of the missing values. Finding the specific regular expression is currently done manually. A memory trace can be described as a univariate time-series, and there are many approaches to imputing missing values in univariate time-series, like Linear Interpolation, Spline Interpolation, and Nearest Neighbour Interpolation. These approaches assume knowledge of which index the data is missing which is not the case in our scenario. Dynamic Time Warping based imputation [21] has shown to be effective at imputing missing data in univariate time-series, which is why we chose this approach. However, we believe there is room for improvement in this area, and we encourage future work to explore this topic further.

Chapter 7

Conclusions

In this thesis, we developed *SpySGX*, a framework that uses a novel data science approach to address the path-explosion problem in symbolic execution with memory traces. This allowed us to infer secrets using memory traces from SGXenclaves automatically. We demonstrated our approach on two example enclaves, *Square-Multiply* and *SubBytes*, and show that it can reverse the secret 100 % of the time if the traces are complete. While Faultmorse [17] can also reverse the secret if the traces follow a specific pattern, we believe we are the first to use symbolic execution. Our approach allows us to have a more general approach that can reverse secrets from instruction Access Attacks and data-access Attacks that do not depend on specific data patterns.

One of the mitigation strategies against microarchitectural side-channel attacks [19] is to minimize the number of memory-accesses observed. To evaluate its effectiveness, we developed two strategies to handle missing data. We developed a Missing instruction Heuristic to select the correct branches in an instruction Access Attack. We also showcase a DTW-based approach to impute missing data in a data-access Attack. With the Missing instruction Heuristic, we show that we can reverse the secret 98 % of the time, even if we miss 25 % of the data-accesses. Our DTW-based approach shows that our imputation method with a window size 1 is within 10 % of a perfect theoretical imputation.

We believe our work will further the research in the automated analysis of memory-based side-channel attacks. An open question is whether our approach can be applied to Real-World enclaves, given the high number of constraints generated by the symbolic execution engine. We also believe there is room for improvements in our Missing instruction Heuristic, given the high number of *Bug* results, showing that theoretically, it could be possible to reverse the secret from traces with even 80 % missing data.

Our work provides professional enclave developers with a new methodology

and tool to verify potential memory side-channel vulnerabilities in *SGX* enclaves. This will save developers valuable time and resources, as they do not have to manually analyze the enclave or fix false positives, resulting in a more secure *SGX* ecosystem. Our novel approach combining symbolic execution with data science can also advance research in automated verification of memory-based side-channel attacks outside of *SGX* enclaves, given that our framework generalizes for multiple architectures.

Bibliography

- [1] V. Costan and S. Devadas, “Intel sgx explained,” *Cryptology ePrint Archive*, 2016.
- [2] S. van Schaik, A. Seto, T. Yurek, A. Batori, B. AlBassam, C. Garman, D. Genkin, A. Miller, E. Ronen, and Y. Yarom, “SoK: SGX.Fail: How stuff get eXposed,” 2022.
- [3] D. Moghimi, J. Van Bulck, N. Heninger, F. Piessens, and B. Sunar, “Copycat: Controlled instruction-level attacks on enclaves,” in *Proceedings of the 29th USENIX Conference on Security Symposium*, 2020, pp. 469–486.
- [4] Y. Xu, W. Cui, and M. Peinado, “Controlled-channel attacks: Deterministic side channels for untrusted operating systems,” in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 640–656.
- [5] W. Wang, G. Chen, X. Pan, Y. Zhang, X. Wang, V. Bindshaedler, H. Tang, and C. A. Gunter, “Leaky cauldron on the dark land: Understanding memory side-channel hazards in sgx,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 2421–2434.
- [6] “Intel sgx official statement on ssde-channel attacks.” [Online]. Available: <https://www.intel.com/content/www/us/en/architecture-and-technology/software-guard-extensions-enhanced-data-protection.html>
- [7] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi, “Verifying constant-time implementations.” in *USENIX Security Symposium*, vol. 16, 2016, pp. 53–70.
- [8] “Libgcrypt, the gnu crypto library.” [Online]. Available: <https://github.com/gpg/libgcrypt>
- [9] J. Van Bulck, N. Weichbrodt, R. Kapitza, F. Piessens, and R. Strackx, “Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution,” in *Proceedings of the 26th USENIX Security Symposium*. USENIX Association, 2017, pp. 1041–1056.

- [10] “Usage statistics of linux for websites.” [Online]. Available: <https://w3techs.com/technologies/details/os-linux>
- [11] “Wolfssl embedded tls library.” [Online]. Available: <https://www.wolfssl.com/>
- [12] S. Weiser, A. Zankl, R. Spreitzer, K. Miller, S. Mangard, and G. Sigl, “DATA - differential address trace analysis: Finding address-based side-channels in binaries,” in *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*, W. Enck and A. P. Felt, Eds. USENIX Association, 2018, pp. 603–620. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/weiser>
- [13] Y. Xiao, M. Li, S. Chen, and Y. Zhang, “Stacco: Differentially analyzing side-channel traces for detecting ssl/tls vulnerabilities in secure enclaves,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 859–874.
- [14] J. Wichelmann, A. Moghimi, T. Eisenbarth, and B. Sunar, “Microwalk: A framework for finding side channels in binaries,” in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 161–173.
- [15] W. Wang and Y. Zhang, “Time and order: Towards automatically identifying side-channel vulnerabilities in enclave binaries,” in *Proceedings of the 22nd International Symposium on Research in Attacks, Intrusions and Defenses*, 2019.
- [16] T. Yavuz, F. Fowze, G. Hernandez, K. Y. Bai, K. Butler, and D. J. Tian, “Encider: Detecting timing and cache side channels in sgx enclaves and cryptographic apis,” *IEEE Transactions on Dependable and Secure Computing*, 2022.
- [17] L. Hu, F. Zhang, Z. Liang, R. Ding, X. Cai, Z. Wang, and W. Jin, “Faultmorse: An automated controlled-channel attack via longest recurring sequence,” *Computers & Security*, vol. 124, p. 103003, 2023.
- [18] J. Van Bulck, F. Piessens, and R. Strackx, “Sgx-step: A practical attack framework for precise enclave execution control,” in *Proceedings of the 2nd Workshop on System Software for Trusted Execution*, 2017, pp. 1–6.
- [19] *Asynchronous Enclave Exit Notify and the EDECCSSA User Leaf Function [White Paper]*, 2022, uRL: <https://cdrdv2-public.intel.com/736463/aex-notify-white-paper-public.pdf> [accessed: 24.04.2023]. [Online]. Available: <https://cdrdv2-public.intel.com/736463/aex-notify-white-paper-public.pdf>

- [20] P. Antonino, W. A. Woloszyn, and A. W. Roscoe, “Guardian: Symbolic validation of orderliness in sgx enclaves,” in *Proceedings of the 2021 on Cloud Computing Security Workshop*, ser. CCSW ’21. New York, NY, USA: Association for Computing Machinery, 11 2021, p. 111–123. [Online]. Available: <https://doi.org/10.1145/3474123.3486755>
- [21] É. P. Caillault, A. Lefebvre, A. Bigand *et al.*, “Dynamic time warping-based imputation for univariate time series data,” *Pattern Recognition Letters*, vol. 139, pp. 139–147, 2020.
- [22] “Pytorch with sgx,” accessed: 2023-03-20. [Online]. Available: https://github.com/intel/sgx-pytorch/tree/sgx/enclave_ops
- [23] M. Schwarz, S. Weiser, D. Gruss, C. Maurice, and S. Mangard, “Malware guard extension: Using sgx to conceal cache attacks,” in *Detection of Intrusions and Malware, and Vulnerability Assessment: 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings 14*. Springer, 2017, pp. 3–24.
- [24] S. Lee, M.-W. Shih, P. Gera, T. Kim, H. Kim, and M. Peinado, “Inferring fine-grained control flow inside sgx enclaves with branch shadowing,” in *USENIX Security Symposium*, vol. 19, 2017, pp. 16–18.
- [25] A. Moghimi, J. Wichelmann, T. Eisenbarth, and B. Sunar, “Memjam: A false dependency attack against constant-time crypto implementations,” *International Journal of Parallel Programming*, vol. 47, pp. 538–570, 2019.
- [26] A. C. Aldaya, B. B. Brumley, S. ul Hassan, C. P. García, and N. Tuveri, “Port contention for fun and profit,” in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 870–887.
- [27] P. Jain, S. Desai, S. Kim, M.-W. Shih, J. Lee, C. Choi, Y. Shin, T. Kim, B. B. Kang, and D. Han, “OpenSGX: An Open Platform for SGX Research,” in *Proceedings of the Network and Distributed System Security Symposium*, San Diego, CA, Feb. 2016.
- [28] “Qemu, a generic and open source machine emulator and virtualizer.” [Online]. Available: <https://www.qemu.org/docs/master/system/i386/sgx.html>
- [29] T. K. Vintsyuk, “Speech discrimination by dynamic programming,” *Cybernetics*, vol. 4, no. 1, pp. 52–57, 1968.
- [30] STEPAN, “The best citation.”

- [31] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, “Sok: (state of) the art of war: Offensive techniques in binary analysis,” 2016.
- [32] “Intel® software guard extensions ssl (<https://github.com/intel/intel-sgx-ssl>) [accessed 22.05.2023].”
- [33] “Python source for pow().” [Online]. Available: <https://github.com/python/cpython/blob/109fc2792a490ee5cd8a423e17d415fbdedec5c8/Objects/longobject.c#L4244-L4447>
- [34] S. Shinde, Z. L. Chua, V. Narayanan, and P. Saxena, “Preventing page faults from telling your secrets,” in *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*, 2016, pp. 317–328.
- [35] F. I. P. S. P. 197, “Announcing the advanced encryption standard (aes),” Nov. 2001. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- [36] “Understanding golang’s aes implementation: T-tables, <https://blog.tclaverie.eu/posts/understanding-golangs-aes-implementation-t-tables/> (accessed: 20.05.2023).”
- [37] S. C. Pohlig and M. E. Hellman, “An improved algorithm for computing logarithms over $gf(p)$ and its cryptographic significance,” in *Democratizing Cryptography: The Work of Whitfield Diffie and Martin Hellman*, 2022, pp. 415–430.
- [38] E. Barker and Q. Dang, “Nist special publication 800-57 part 1, revision 4,” *NIST, Tech. Rep*, vol. 16, 2016.
- [39] “Tinyaes, small portable aes128/192/256 in c.” [Online]. Available: <https://github.com/kokke/tiny-AES-c>
- [40] S. A. Cook, “The complexity of theorem-proving procedures,” in *Proceedings of the third annual ACM symposium on Theory of computing*, 1971, pp. 151–158.
- [41] T. Cloosters, M. Rodler, and L. Davi, “{TeeRex}: Discovery and exploitation of memory corruption vulnerabilities in {SGX} enclaves,” in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 841–858.
- [42] T. Kubota, K. Yoshida, M. Shiozaki, and T. Fujino, “Deep learning side-channel attack against hardware implementations of aes,” *Microprocessors and Microsystems*, vol. 87, p. 103383, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0141933120305408>